

PORTADA

Análisis y Diseño de Sistemas II

Semana 16 — Pruebas Unitarias: TDD y Buenas Prácticas

Universidad de Antioquía · Ingeniería de Sistemas



APERTURA

**¿Y si la prueba se escribe
antes que el código?**

TRANSICIÓN

De probar lo que ya existe, a diseñar probando

El martes se vio pruebas unitarias como una herramienta de **verificación**: se escribe código, y luego se escribe una prueba que confirma que funciona. Hoy se ve un enfoque distinto, donde la prueba no viene después del código, sino **antes**: **TDD** (Test-Driven Development, desarrollo guiado por pruebas), creado por **Kent Beck** — el mismo coautor de JUnit visto la sesión pasada.

TDD no es solo una técnica de pruebas: es una técnica de **diseño**. Escribir la prueba primero cambia cómo se piensa el código, y conecta directamente con SOLID y DIP, vistos en la Unidad 4.

CONCEPTO

Qué es TDD

Test-Driven Development (TDD) es una práctica de desarrollo en la que se escribe una prueba unitaria **antes** de escribir el código de producción que la hace pasar. Kent Beck la formalizó y popularizó a principios de los años 2000, como parte de las prácticas de **Extreme Programming (XP)**, aunque el propio Beck reconoce haber "redescubierto" una práctica que ya circulaba de forma informal en los primeros equipos de programación.

La idea central: en vez de escribir código y después preguntarse "¿cómo pruebo esto?", primero se pregunta "¿cómo se vería una prueba que confirme que esto funciona?" — y esa pregunta se responde escribiendo la prueba antes de que el código exista.

CONCEPTO

El ciclo Red-Green-Refactor

TDD se ejecuta en un ciclo corto y repetido, con tres pasos que le dan nombre:

- **Red (rojo):** se escribe una prueba unitaria para una funcionalidad que todavía no existe. La prueba falla — no puede ser de otra forma, porque el código aún no está escrito.
- **Green (verde):** se escribe el código de producción **mínimo** necesario para que esa prueba pase. No se busca la solución elegante todavía, solo que la prueba pase.
- **Refactor:** con la prueba en verde como red de seguridad, se mejora la estructura interna del código —aplicando lo visto la semana pasada— sin romper la prueba.

El ciclo se repite, prueba por prueba, hasta completar la funcionalidad — cada vuelta agrega un incremento pequeño y verificado.

CONCEPTO

Por qué escribir la prueba primero cambia el diseño

Escribir la prueba antes de la implementación obliga a pensar primero en la **interfaz** —cómo se va a llamar la función, qué recibe, qué devuelve— antes que en el **algoritmo interno** que la resuelve. Esto tiene un efecto de diseño concreto: para que algo sea fácil de probar de forma aislada, tiene que ser fácil de aislar de sus dependencias.

Código difícil de probar casi siempre es código con **alto acoplamiento** —una clase que depende directamente de una base de datos concreta, o de otra clase con muchas responsabilidades. TDD, sin proponérselo explícitamente, empuja hacia el mismo tipo de diseño que ya perseguían Low Coupling (GRASP) y DIP (SOLID).

CONTRAEJEMPLO

Un ciclo TDD completo, paso a paso

```
// RED: la prueba existe, la función aun no
def test_debe_rechazar_deposito_con_monto_negativo():
    cuenta = CuentaBancaria(saldo=100)
    with pytest.raises(ValueError):
        cuenta.depositar(-50)
# Falla: CuentaBancaria.depositar no existe todavia

// GREEN: código mínimo para que pase
class CuentaBancaria:
    def __init__(self, saldo):
        self.saldo = saldo
    def depositar(self, monto):
        if monto < 0:
            raise ValueError("monto invalido")
        self.saldo += monto
# La prueba pasa

// REFACTOR: con la prueba en verde, se puede mejorar
# (ej. extraer la validación a un método propio) sin miedo
```

CONCEPTO

Buenas prácticas de pruebas (I)

- **Una aserción lógica por prueba:** cada prueba debería verificar un solo comportamiento. Varias aserciones sueltas y sin relación en la misma prueba dificultan saber, cuando falla, cuál de todas era el problema real.
- **Nombres descriptivos:** el nombre de la prueba debería leerse casi como una frase que describe el comportamiento esperado — ej. `debe_devolver_error_cuando_el_monto_es_negativo` comunica mucho más que `test1` o `testDepositar` .
- **Evitar pruebas frágiles acopladas a la implementación:** una prueba no debería fallar porque se renombró una variable interna o se reordenó el código, solo porque cambió el **comportamiento observable**.



CONCEPTO

Buenas prácticas de pruebas (II): cobertura

La **cobertura de código** (code coverage) mide qué porcentaje de las líneas o ramas del código se ejecutan al correr la suite de pruebas. Es una métrica **útil pero no suficiente**: un 100% de cobertura no garantiza que las pruebas verifiquen realmente el comportamiento correcto, solo que el código se ejecutó al menos una vez.

Es perfectamente posible tener 100% de cobertura con pruebas que no tienen ninguna aserción significativa — cubren la línea, pero no verifican nada real. Por eso **perseguir 100% de cobertura de forma ciega** es un objetivo equivocado: la meta es cubrir los casos que importan (incluyendo casos límite y de error), no maximizar un número.

TENSIÓN CONCEPTUAL

¿TDD siempre, o solo cuando aporta?

Kent Beck y la comunidad de XP defienden TDD como práctica sistemática para todo código de producción. En la práctica de la industria, esa postura no es unánime: hay desarrolladores expertos —incluido el propio **Robert C. Martin**, defensor entusiasta de TDD en su libro *Clean Code*, en tensión con voces como la de **David Heinemeier Hansson** (creador de Ruby on Rails), crítico público de TDD como dogma— que cuestionan escribir la prueba primero en **todos** los casos, especialmente en exploración temprana de un diseño incierto.

Lo que sí tiene consenso amplio: sin importar el orden en que se escriban prueba y código, **terminar con buena cobertura de pruebas automatizadas es innegociable** para cualquier proyecto que se vaya a mantener en el tiempo.

INTERACCIÓN

Escriban el primer paso de un ciclo TDD

En el chat, para la función `calcularEdadMinima(fechaNacimiento)` —que debe validar que una persona tenga al menos 18 años— escriban:

1. El **nombre** de una prueba unitaria (siguiendo la convención descriptiva vista hoy) para el caso "persona menor de edad".
2. Qué esperarían que devuelva o lance esa función en ese caso (paso **Red**).

5 minutos · respondan en el chat

SÍNTESIS

Ideas clave de la sesión

1. TDD (Kent Beck) escribe la prueba **antes** que el código de producción.
2. El ciclo es **Red → Green → Refactor**, repetido en incrementos pequeños.
3. Escribir la prueba primero fuerza a pensar en la interfaz antes que en la implementación, empujando hacia bajo acoplamiento — el mismo objetivo de DIP y Low Coupling.
4. Una aserción lógica por prueba, nombres descriptivos y evitar acoplar la prueba a detalles de implementación son buenas prácticas centrales.
5. La cobertura de código es útil, pero perseguir 100% de forma ciega no garantiza pruebas de calidad.



CIERRE

Antes de la próxima sesión

Confirмен con su equipo cuáles 3 métodos o funciones clave de su proyecto de Fábrica Escuela van a probar en el laboratorio del viernes, y aseguren tener listo el entorno del framework de pruebas de su stack (JUnit 5, xUnit/NUnit, pytest o Jest). El viernes se cierra la **Unidad 6** completa con el **laboratorio de pruebas unitarias sobre el proyecto**.

